

QUESTION 1

Problem Statement

Design an **Employee Salary Management System** using Object-Oriented Programming concepts.

The system should handle different types of employees and calculate their salaries based on their roles. The program must demonstrate **inheritance, constructor chaining using super(), method overriding, dynamic method dispatch, and exception handling.**

Requirements

Base Class: Employee

- Data Members:
 - String name
 - double salary
- Constructor:
 - Initialize values using constructor
 - If salary < 0 → throw exception
- Method:

double calculateSalary()

Child Class 1: Manager

- Inherits Employee
 - Gets **20% bonus**
 - Override calculateSalary()
-

Child Class 2: Developer

- Inherits Employee
- Extra field:
 - int overtimeHours
- Each hour = ₹200
- Validate overtime (≥ 0)
- Override method

Constraints

- $1 \leq n \leq 100$
 - $0 \leq \text{salary} \leq 10^7$
 - $0 \leq \text{overtime} \leq 100$
-

Input Format

n

Type Name Salary [Overtime]

- Type:
 - M → Manager
 - D → Developer
-

Output Format

Final Salary of <Name>: <Amount>

OR

Error: <message>

Sample Input

3

M Roy 50000

D Aman 40000 5

D Rahul -1000 2

Sample Output

Final Salary of Roy: 60000.0

Final Salary of Aman: 41000.0

Error: Invalid Salary

Code

```
import java.util.*;
```

```
class Employee {  
    String name;  
    double salary;
```

```

Employee(String name, double salary) {
    if (salary < 0) {
        throw new IllegalArgumentException("Invalid Salary");
    }
    this.name = name;
    this.salary = salary;
}

double calculateSalary() {
    return salary;
}
}

class Manager extends Employee {

    Manager(String name, double salary) {
        super(name, salary); // constructor chaining
    }

    @Override
    double calculateSalary() {
        return salary + 0.2 * salary;
    }
}

class Developer extends Employee {
    int overtimeHours;

    Developer(String name, double salary, int overtimeHours) {
        super(name, salary);

        if (overtimeHours < 0) {
            throw new IllegalArgumentException("Invalid Overtime");
        }

        this.overtimeHours = overtimeHours;
    }

    @Override
    double calculateSalary() {
        return salary + overtimeHours * 200;
    }
}

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
    }
}

```

```
int n = sc.nextInt();

while (n-- > 0) {
    try {
        String type = sc.next();
        String name = sc.next();
        double salary = sc.nextDouble();

        Employee emp; // dynamic dispatch

        if (type.equals("M")) {
            emp = new Manager(name, salary);
        } else {
            int overtime = sc.nextInt();
            emp = new Developer(name, salary, overtime);
        }

        System.out.println("Final Salary of " + name + ": " + emp.calculateSalary());

    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}
}
```

ALL POSSIBLE TEST CASES

1. Normal Manager

Input:

1

M Roy 50000

Output:

Final Salary of Roy: 60000.0

2. Normal Developer

Input:

1

D Aman 40000 10

Output:

Final Salary of Aman: 42000.0

3. Negative Salary

Input:

1

M Rahul -5000

Output:

Error: Invalid Salary

4. Negative Overtime

Input:

1

D Karan 30000 -2

Output:

Error: Invalid Overtime

5. Zero Salary

Input:

1

M Zero 0

Output:

Final Salary of Zero: 0.0

6. Zero Overtime

Input:

1

D Dev 30000 0

Output:

Final Salary of Dev: 30000.0

7. Mixed Case

Input:

4

M A 50000

D B 40000 10

M C -1000

D D 20000 -5

Output:

Final Salary of A: 60000.0

Final Salary of B: 42000.0

Error: Invalid Salary

Error: Invalid Overtime

QUESTION 2: BANKING SYSTEM WITH INTERFACE

Problem Statement

Design a Banking System using Object-Oriented Programming concepts.

The system should support different types of bank accounts and calculate the final balance based on account-specific rules. The program must demonstrate **interface usage, inheritance, constructor chaining using super(), method overriding, dynamic method dispatch, and exception handling.**

Requirements

Interface: AccountOperations

- Method:

double calculateBalance();

Base Class: BankAccount

- Implements AccountOperations
 - Data Members:
 - String name
 - double balance
 - Constructor:
 - Initialize values
 - If balance < 0 → throw exception
-

Child Class 1: SavingsAccount

- Inherits BankAccount
- Interest = 5%

- Override calculateBalance()
-

Child Class 2: CurrentAccount

- Inherits BankAccount
 - If balance < 1000 → penalty ₹100
 - Override calculateBalance()
-

Constraints

- $1 \leq n \leq 100$
 - $0 \leq \text{balance} \leq 10^7$
-

Input Format

n

Type Name Balance

- Type:
 - S → Savings Account
 - C → Current Account
-

Output Format

Final Balance of <Name>: <Amount>

OR

Error: <message>

Sample Input

3

S Roy 10000

C Aman 800

C Rahul -2000

Sample Output

Final Balance of Roy: 10500.0

Final Balance of Aman: 700.0

Error: Invalid Balance

Code

```
import java.util.*;

interface AccountOperations {
    double calculateBalance();
}

class BankAccount implements AccountOperations {
    String name;
    double balance;

    BankAccount(String name, double balance) {
        if (balance < 0) {
            throw new IllegalArgumentException("Invalid Balance");
        }
        this.name = name;
        this.balance = balance;
    }

    public double calculateBalance() {
        return balance;
    }
}

class SavingsAccount extends BankAccount {

    SavingsAccount(String name, double balance) {
        super(name, balance);
    }

    @Override
    public double calculateBalance() {
        return balance + (0.05 * balance);
    }
}

class CurrentAccount extends BankAccount {

    CurrentAccount(String name, double balance) {
        super(name, balance);
    }

    @Override
    public double calculateBalance() {
        if (balance < 1000) {
            return balance - 100;
        }
    }
}
```

```

        return balance;
    }
}

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        while (n-- > 0) {
            try {
                String type = sc.next();
                String name = sc.next();
                double balance = sc.nextDouble();

                AccountOperations acc; // dynamic dispatch using interface

                if (type.equals("S")) {
                    acc = new SavingsAccount(name, balance);
                } else {
                    acc = new CurrentAccount(name, balance);
                }

                System.out.println("Final Balance of " + name + ": " + acc.calculateBalance());

            } catch (Exception e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }
}

```

All Possible Test Cases

1. Normal Savings Account

Input:

1

S Roy 10000

Output:

Final Balance of Roy: 10500.0

2. Normal Current Account

Input:

1

C Aman 5000

Output:

Final Balance of Aman: 5000.0

3. Current Account with Penalty

Input:

1

C Aman 800

Output:

Final Balance of Aman: 700.0

4. Zero Balance

Input:

1

S Zero 0

Output:

Final Balance of Zero: 0.0

5. Negative Balance

Input:

1

C Rahul -1000

Output:

Error: Invalid Balance

6. Multiple Mixed Inputs

Input:

4

S A 10000

C B 800

S C 2000

C D -500

Output:

Final Balance of A: 10500.0
Final Balance of B: 700.0
Final Balance of C: 2100.0
Error: Invalid Balance

QUESTION 3: SHOPPING BILLING SYSTEM

Problem Statement

Design a Shopping Billing System using Object-Oriented Programming concepts.

The system should handle different types of products and calculate their final price based on category-specific rules. The program must demonstrate **inheritance, constructor chaining using super(), method overriding, dynamic method dispatch, and exception handling.**

Requirements

Base Class: Product

- Data Members:
 - String name
 - double price
- Constructor:
 - Initialize values
 - If price < 0 → throw exception
- Method:

double finalPrice()

Child Class 1: Electronics

- GST = 18%
 - Override method
-

Child Class 2: Clothing

- Discount = 10%
 - Override method
-

Constraints

- $1 \leq n \leq 100$
 - $0 \leq \text{price} \leq 10^7$
-

Input Format

n

Type Name Price

- Type:
 - E → Electronics
 - C → Clothing
-

Output Format

Final Price of <Name>: <Amount>

OR

Error: <message>

Sample Input

3

E Laptop 50000

C Shirt 2000

E Phone -10000

Sample Output

Final Price of Laptop: 59000.0

Final Price of Shirt: 1800.0

Error: Invalid Price

Code

```
import java.util.*;
```

```
class Product {  
    String name;
```

```

double price;

Product(String name, double price) {
    if (price < 0) {
        throw new IllegalArgumentException("Invalid Price");
    }
    this.name = name;
    this.price = price;
}

double finalPrice() {
    return price;
}
}

class Electronics extends Product {

    Electronics(String name, double price) {
        super(name, price); // constructor chaining
    }

    @Override
    double finalPrice() {
        return price + (0.18 * price);
    }
}

class Clothing extends Product {

    Clothing(String name, double price) {
        super(name, price);
    }

    @Override
    double finalPrice() {
        return price - (0.10 * price);
    }
}

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        while (n-- > 0) {
            try {
                String type = sc.next();
            }
        }
    }
}

```

```
String name = sc.next();
double price = sc.nextDouble();

Product p; // dynamic dispatch

if (type.equals("E")) {
    p = new Electronics(name, price);
} else {
    p = new Clothing(name, price);
}

System.out.println("Final Price of " + name + ": " + p.finalPrice());

} catch (Exception e) {
    System.out.println("Error: " + e.getMessage());
}
}
}
```

All Possible Test Cases

1. Normal Electronics

Input:

1

E Laptop 50000

Output:

Final Price of Laptop: 59000.0

2. Normal Clothing

Input:

1

C Shirt 2000

Output:

Final Price of Shirt: 1800.0

3. Price = 0

Input:

1

E Item 0

Output:
Final Price of Item: 0.0

4. Negative Price

Input:
1
C Jacket -500

Output:
Error: Invalid Price

5. Mixed Input

Input:
4
E A 1000
C B 2000
E C -100
C D 500

Output:

Final Price of A: 1180.0
Final Price of B: 1800.0
Error: Invalid Price
Final Price of D: 450.0

QUESTION 4: STUDENT RESULT SYSTEM

Problem Statement

Design a Student Result System using Object-Oriented Programming concepts.

The system should evaluate student performance and assign grades based on different rules for undergraduate and postgraduate students. The program must demonstrate **inheritance, constructor chaining using super(), method overriding, dynamic method dispatch, and exception handling**.

Requirements

Base Class: Student

- Data Members:
 - String name
 - int marks
- Constructor:
 - Initialize values
 - If marks < 0 OR marks > 100 → throw exception
- Method:

String getGrade()

Child Class 1: Undergraduate

- Grading Rules:
 - $\geq 75 \rightarrow$ Distinction
 - $\geq 50 \rightarrow$ Pass
 - $< 50 \rightarrow$ Fail
 - Override method
-

Child Class 2: Postgraduate

- Grading Rules:
 - $\geq 80 \rightarrow$ Distinction
 - $\geq 60 \rightarrow$ Pass
 - $< 60 \rightarrow$ Fail
 - Override method
-

Constraints

- $1 \leq n \leq 100$
 - $0 \leq \text{marks} \leq 100$
-

Input Format

n

Type Name Marks

- Type:

- U → Undergraduate
- P → Postgraduate

Output Format

<Name> Grade: <Grade>

OR

Error: <message>

Sample Input

3
U A 78
P B 65
U C 120

Sample Output

A Grade: Distinction
B Grade: Pass
Error: Invalid Marks

Code

```
import java.util.*;

class Student {
    String name;
    int marks;

    Student(String name, int marks) {
        if (marks < 0 || marks > 100) {
            throw new IllegalArgumentException("Invalid Marks");
        }
        this.name = name;
        this.marks = marks;
    }

    String getGrade() {
        return "";
    }
}

class Undergraduate extends Student {
```

```

    Undergraduate(String name, int marks) {
        super(name, marks); // constructor chaining
    }

    @Override
    String getGrade() {
        if (marks >= 75) return "Distinction";
        else if (marks >= 50) return "Pass";
        else return "Fail";
    }
}

class Postgraduate extends Student {

    Postgraduate(String name, int marks) {
        super(name, marks);
    }

    @Override
    String getGrade() {
        if (marks >= 80) return "Distinction";
        else if (marks >= 60) return "Pass";
        else return "Fail";
    }
}

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        while (n-- > 0) {
            try {
                String type = sc.next();
                String name = sc.next();
                int marks = sc.nextInt();

                Student s; // dynamic dispatch

                if (type.equals("U")) {
                    s = new Undergraduate(name, marks);
                } else {
                    s = new Postgraduate(name, marks);
                }

                System.out.println(name + " Grade: " + s.getGrade());
            }
        }
    }
}

```

```
        } catch (Exception e) {  
            System.out.println("Error: " + e.getMessage());  
        }  
    }  
}  
}
```

All Possible Test Cases

1. Undergraduate Distinction

Input:

1

U A 80

Output:

A Grade: Distinction

2. Undergraduate Pass

Input:

1

U B 60

Output:

B Grade: Pass

3. Undergraduate Fail

Input:

1

U C 40

Output:

C Grade: Fail

4. Postgraduate Distinction

Input:

1

P D 85

Output:

D Grade: Distinction

5. Postgraduate Pass

Input:

1

P E 65

Output:

E Grade: Pass

6. Postgraduate Fail

Input:

1

P F 50

Output:

F Grade: Fail

7. Invalid Marks (>100)

Input:

1

U G 120

Output:

Error: Invalid Marks

8. Invalid Marks (negative)

Input:

1

P H -10

Output:

Error: Invalid Marks

9. Mixed Input

Input:

4

U A 75

P B 80

U C 30

P D 110

Output:

A Grade: Distinction

B Grade: Distinction

C Grade: Fail

Error: Invalid Marks

QUESTION 5: VEHICLE INSURANCE SYSTEM

Problem Statement

Design a Vehicle Insurance System using Object-Oriented Programming concepts.

The system should calculate the insurance amount for different types of vehicles based on their price and specific rules. The program must demonstrate **inheritance, constructor chaining using super(), method overriding, dynamic method dispatch, and exception handling**.

Requirements

Base Class: Vehicle

- Data Members:
 - String brand
 - double price
- Constructor:
 - Initialize values
 - If price < 0 → throw exception
- Method:

double calculateInsurance()

Child Class 1: Car

- Insurance = 8% of price
 - If price > 10,00,000 → extra ₹5000 added
 - Override method
-

Child Class 2: Bike

- Insurance = 5% of price
 - If price < 50,000 → minimum insurance ₹2000
 - Override method
-

Constraints

- $1 \leq n \leq 100$
 - $0 \leq \text{price} \leq 10^7$
-

Input Format

n

Type Brand Price

- Type:
 - C → Car
 - B → Bike
-

Output Format

Insurance for <Brand>: <Amount>

OR

Error: <message>

Sample Input

3

C Honda 1200000

B Hero 40000

C BMW -500000

Sample Output

Insurance for Honda: 101000.0

Insurance for Hero: 2000.0

Error: Invalid Price

Code

```
import java.util.*;
```

```
class Vehicle {
    String brand;
    double price;

    Vehicle(String brand, double price) {
        if (price < 0) {
            throw new IllegalArgumentException("Invalid Price");
        }
        this.brand = brand;
        this.price = price;
    }

    double calculateInsurance() {
        return 0;
    }
}
```

```
class Car extends Vehicle {

    Car(String brand, double price) {
        super(brand, price); // constructor chaining
    }

    @Override
    double calculateInsurance() {
        double insurance = price * 0.08;
        if (price > 1000000) {
            insurance += 5000;
        }
        return insurance;
    }
}
```

```
class Bike extends Vehicle {

    Bike(String brand, double price) {
        super(brand, price);
    }

    @Override
    double calculateInsurance() {
        double insurance = price * 0.05;
        if (price < 50000) {
            return 2000;
        }
        return insurance;
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        while (n-- > 0) {
            try {
                String type = sc.next();
                String brand = sc.next();
                double price = sc.nextDouble();

                Vehicle v; // dynamic dispatch

                if (type.equals("C")) {
                    v = new Car(brand, price);
                } else {
                    v = new Bike(brand, price);
                }

                System.out.println("Insurance for " + brand + ": " + v.calculateInsurance());

            } catch (Exception e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }
}
```

All Possible Test Cases

1. Normal Car

Input:

1

C Honda 800000

Output:

Insurance for Honda: 64000.0

2. Car with Extra Charge

Input:

1

C BMW 1200000

Output:

Insurance for BMW: 101000.0

3. Normal Bike

Input:

1

B Yamaha 100000

Output:

Insurance for Yamaha: 5000.0

4. Low Price Bike (Minimum Rule)

Input:

1

B Hero 40000

Output:

Insurance for Hero: 2000.0

5. Price = 0

Input:

1

C Test 0

Output:

Insurance for Test: 0.0

6. Negative Price

Input:

1

B Test -1000

Output:

Error: Invalid Price

7. Mixed Input

Input:

4

C A 1200000

B B 40000

C C 500000

B D -100

Output:

Insurance for A: 101000.0

Insurance for B: 2000.0

Insurance for C: 40000.0

Error: Invalid Price